

**Lab 1: Requirements Engineering and
UML Use-Case Modelling**

Problem Statement 41: OpenAPI Mock Server Generator

Name	Sidhant Holla
SRN	PES1UG24AM466
Section	H

Requirements Table

Problem Statement 41: OpenAPI Mock Server Generator

Sidhant Holla, PES1UG24AM466, Section H

Scenario: A developer productivity tool that ingests OpenAPI 3.0 YAML or JSON specifications, creates dynamic mock REST endpoints with schema validation, and simulates configurable response latency.

Actors: API Developer, QA Engineer, DevOps Engineer

Functional Requirements

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall parse an uploaded OpenAPI 3.0 specification in YAML or JSON and generate mock HTTP endpoints for every path and operation declared in it, returning randomly generated JSON that matches the declared response schema.	High	Pass: for a spec with 10 paths, all 10 endpoints respond and each response body matches its schema. Fail: a declared path returns 404, or a path that is not in the spec returns 200 OK.	Front-end and consumer teams can start work before the real API is ready.
FR-002	Functional	The system shall check the uploaded specification against the OpenAPI 3.0 schema before generating anything, and reject an invalid file with a list of errors giving the line number and a readable message.	High	Pass: a spec with no info.title is rejected and the error names info.title and its line number. Fail: an invalid spec is accepted, or the error message gives no location.	Stops a broken mock server from being built and tells the developer what to fix.
FR-003	Functional	The system shall check every incoming request (path parameters, query parameters, headers and body) against the schema for that operation, and return HTTP 400 listing the fields that failed when the check does not pass.	High	Pass: a POST that sends text in an integer field returns 400 and names that field. Fail: the request returns any 2xx status.	Makes the mock enforce the API contract, so client-side bugs show up early.
FR-004	Functional	The system shall let the user set a response delay between 0 and 10000 ms for each endpoint, along with a fault rule giving an HTTP status code and the percentage of requests that should receive it. Changes shall take effect without restarting the mock server.	Medium	Pass: with a 500 ms delay and a 20% chance of 503 configured, measured delay over 1000 requests is 490 to 510 ms and 503 is returned for 17 to 23% of them, without a restart. Fail: the setting needs a restart or has no effect.	Lets testers reproduce slow networks and partial outages so timeout and retry logic can be tested.
FR-005	Functional	The system shall log every request and response handled by the mock server with the time, method, path, status code, response time and validation result, and shall let the user filter the log and export it as CSV or JSON.	Medium	Pass: after 50 requests the log has 50 rows, filtering on status 400 shows only those rows, and the exported file has the same number of rows. Fail: rows are missing, or the filter returns the wrong rows.	Gives testers evidence they can attach to a bug report.

Non-Functional Requirements

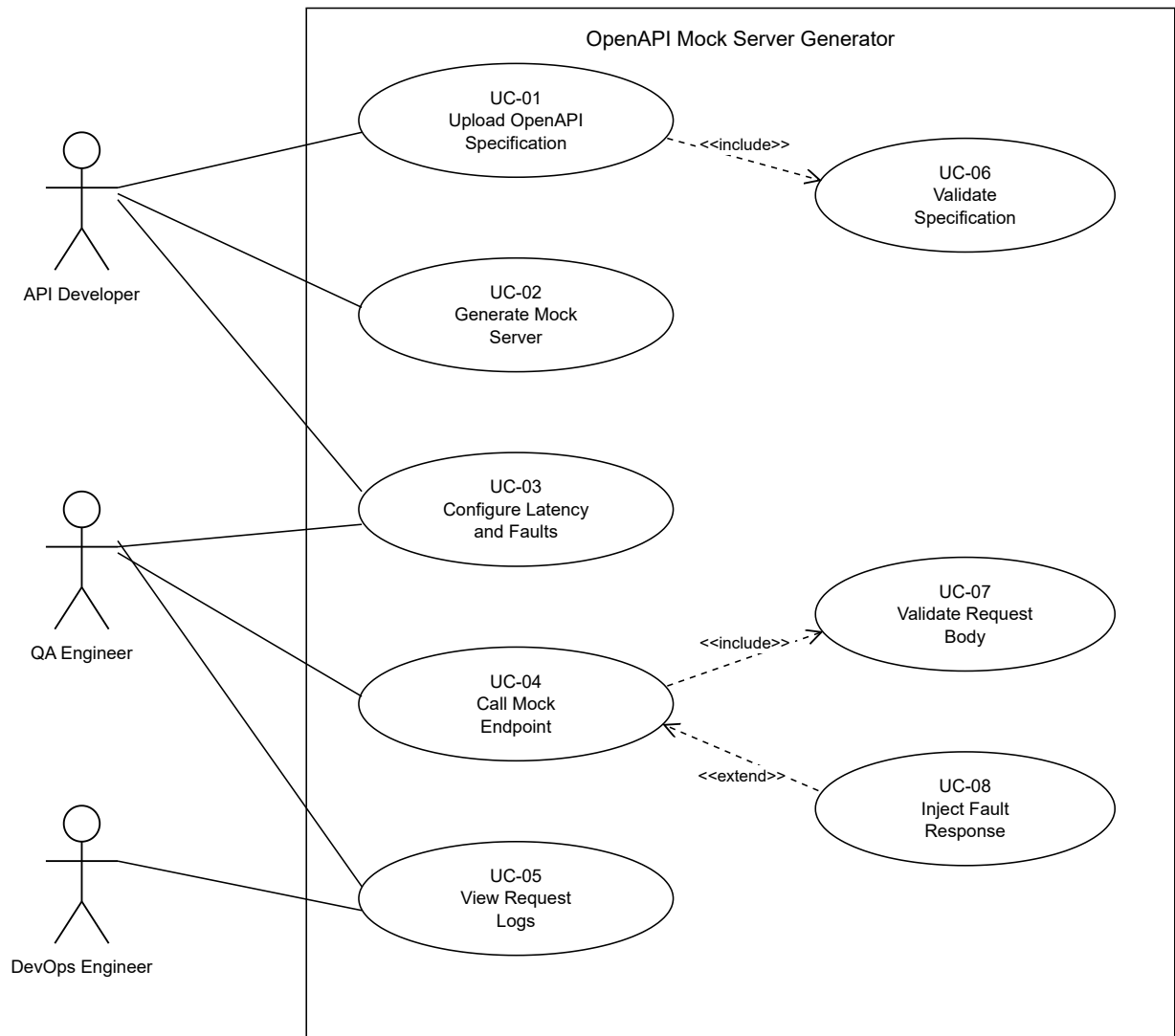
Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
--------	------	-------------	----------	---------------------	-----------

NFR-001	Non-functional (Performance)	The mock server shall handle 1000 requests per second for 5 minutes without dropping connections, and the simulated delay shall stay within 10 ms of the configured value.	High	Pass: a JMeter run at 1000 requests per second for 5 minutes shows no dropped connections, an error rate below 0.1%, and measured delay within 10 ms of the setting. Fail: throughput falls below 1000 per second, or the delay drifts by more than 10 ms.	The mock is used inside automated test runs, so it must not become the bottleneck.
NFR-002	Non-functional (Security)	The system shall require a valid login token to upload, generate, configure or delete a mock server, shall use HTTPS with TLS 1.2 or above, and shall store uploaded specifications in encrypted form.	High	Pass: an upload request without a token returns 401, a TLS scan reports nothing below TLS 1.2, and the stored specification is not readable as plain text in the database. Fail: any of these operations works without a token, or the specification is stored in plain text.	Specifications describe internal APIs that have not been released, so a leak would expose the real system.

UML Use-Case Diagram

Problem Statement 41: OpenAPI Mock Server Generator

Sidhant Holla, PES1UG24AM466, Section H



Use-Case Flow Specification

Problem Statement 41: OpenAPI Mock Server Generator

Sidhant Holla, PES1UG24AM466, Section H

UC-02: Generate Mock Server

Use case ID	UC-02
Use case name	Generate Mock Server from an OpenAPI 3.0 specification
Primary actor	API Developer
Secondary actors	QA Engineer, DevOps Engineer
Related requirements	FR-001, FR-002, NFR-001, NFR-002
Included use case	UC-06 Validate Specification (<<include>>)
Trigger	The API Developer selects Generate Mock Server for an uploaded specification.

Preconditions

1. The API Developer is logged in with a valid token.
2. An OpenAPI 3.0 specification has been uploaded through UC-01.
3. At least one port is free in the mock server port range.

Postconditions

On success

1. A mock server is running on its own base URL, with one endpoint for every path and operation in the specification.
2. Each endpoint returns randomly generated JSON that matches its declared response schema.
3. The specification is stored in encrypted form and the action is written to the audit log.

On failure

1. No mock server is started and no port is used up.
2. The specification stays in the workspace and is still marked as not generated.
3. The validation errors are saved so the developer can look at them later.

Main Success Scenario

1. The API Developer opens the uploaded specification and selects Generate Mock Server.
2. The system shows a form for the base path, the port, the default response delay and the data seed.
3. The API Developer keeps the default values or enters new ones and confirms.
4. The system runs UC-06 Validate Specification to check the file against the OpenAPI 3.0 schema.
5. Validation passes and the system reports how many paths, operations and schemas it found.
6. The system builds a route table mapping each path and HTTP method to its request and response schemas.
7. For each operation the system builds a response generator that produces values matching the declared types, formats, enum values and required fields.
8. The system takes a free port, starts the mock server and sends a test request to one of the new endpoints.
9. The test request returns a valid response and the system marks the server as running.
10. The system shows the base URL, the list of endpoints and a sample cURL command, and writes an audit log entry.
11. The use case ends successfully.

Alternate Flow: 4a. Specification fails validation

- 4a1. UC-06 finds one or more errors, for example a missing info.title or a \$ref that cannot be resolved.
- 4a2. The system stops before taking a port.
- 4a3. The system lists each error with its line number and message and highlights those lines in the editor.
- 4a4. The API Developer either fixes the specification and goes back to step 4, or cancels.
- 4a5. If the developer cancels, the specification stays marked as not generated and the use case ends in failure.